

Building the Lattice Manager - Second Component

Overview

The Lattice Manager defines the D2Q9 (2D, 9-velocity) lattice structure, which is the foundation of the Lattice Boltzmann Method. This component provides:

- 9 lattice velocity directions
- Corresponding weights
- Opposite direction mapping (for bounce-back)
- Equilibrium distribution computation
- Macroscopic quantity extraction

File to Create: **lattice.py**

```
"""D2Q9 Lattice for 2D Lattice Boltzmann Method.
```

```
This module defines the D2Q9 lattice structure with 9 velocity directions,  
providing equilibrium distributions and macroscopic quantity computations.  
"""
```

```
import numpy as np  
from typing import Tuple
```

```
class D2Q9Lattice:  
    """D2Q9 lattice for 2D Lattice Boltzmann Method.
```

```
The D2Q9 lattice has 9 velocity directions:  
- 1 rest particle (0,0)  
- 4 cardinal directions ( $\pm 1, 0$ ) and ( $0, \pm 1$ )  
- 4 diagonal directions ( $\pm 1, \pm 1$ )
```

```
Attributes:
```

```
    c: Lattice velocities (9, 2) - [x, y] components  
    w: Lattice weights (9,) - normalized to sum to 1  
    opp: Opposite directions (9,) - indices for bounce-back  
    cs: Speed of sound ( $1/\sqrt{3}$ )  
    cs2: Speed of sound squared ( $1/3$ )  
    """
```

```
def __init__(self):  
    """Initialize D2Q9 lattice structure."""  
    # Lattice velocities (9 directions)
```

```

# Index layout:
#   6   2   5
#   ↖  ↑  ↗
#   3 ← 0 → 1
#   ↙  ↓  ↘
#   7   4   8
self.c = np.array([
    [0, 0], # 0: rest
    [1, 0], # 1: east
    [0, 1], # 2: north
    [-1, 0], # 3: west
    [0, -1], # 4: south
    [1, 1], # 5: northeast
    [-1, 1], # 6: northwest
    [-1, -1], # 7: southwest
    [1, -1] # 8: southeast
], dtype=float)

# Lattice weights (normalized)
# w[0] = 4/9 for rest particle
# w[1-4] = 1/9 for cardinal directions
# w[5-8] = 1/36 for diagonal directions
self.w = np.array([
    4/9, # 0: rest
    1/9, # 1-4: cardinal
    1/9,
    1/9,
    1/9,
    1/36, # 5-8: diagonal
    1/36,
    1/36,
    1/36
])

# Opposite directions (for bounce-back boundary condition)
# opp[i] gives the index of the direction opposite to i
self.opp = np.array([0, 3, 4, 1, 2, 7, 8, 5, 6])

# Speed of sound in lattice units
self.cs = 1.0 / np.sqrt(3.0)
self.cs2 = self.cs**2

# Verify lattice properties
self._verify_lattice_properties()

def _verify_lattice_properties(self):
    """Verify lattice satisfies required properties."""
    # Check weights sum to 1
    weight_sum = np.sum(self.w)
    assert abs(weight_sum - 1.0) < 1e-12, f"Weights sum to

```

```

{weight_sum}, not 1.0"

# Check opposite directions
for i in range(9):
    opp_i = self.opp[i]
    assert np.allclose(self.c[i], -self.c[opp_i]), \
        f"Direction {i} and opposite {opp_i} don't cancel"

print("✓ D2Q9 lattice properties verified")

def equilibrium(self, rho: np.ndarray, u: np.ndarray, v: np.ndarray) ->
np.ndarray:
    """Compute equilibrium distribution function.

    The equilibrium distribution is given by:
    
$$f_i^{eq} = w_i * \rho * [1 + (c_i \cdot u)/cs^2 + (c_i \cdot u)^2/(2cs^4) - u^2/(2cs^2)]$$


    Args:
        rho: Density field (Nx, Ny)
        u: x-velocity component (Nx, Ny)
        v: y-velocity component (Nx, Ny)

    Returns:
        f_eq: Equilibrium distributions (Nx, Ny, 9)

    Example:
    >>> lattice = D2Q9Lattice()
    >>> rho = np.ones((10, 10))
    >>> u = 0.1 * np.ones((10, 10))
    >>> v = np.zeros((10, 10))
    >>> f_eq = lattice.equilibrium(rho, u, v)
    >>> f_eq.shape
    (10, 10, 9)
    """
    Nx, Ny = rho.shape
    f_eq = np.zeros((Nx, Ny, 9))

    # Velocity squared
    u2 = u**2 + v**2

    # Compute equilibrium for each direction
    for i in range(9):
        # Lattice velocity dotted with macroscopic velocity
        cu = self.c[i, 0] * u + self.c[i, 1] * v

        # Equilibrium distribution
        f_eq[:, :, i] = self.w[i] * rho * (
            1.0
            + cu / self.cs2
            + cu**2 / (2 * self.cs2**2)

```

```

        - u2 / (2 * self.cs2)
    )

    return f_eq

def compute_macroscopic(self, f: np.ndarray) -> Tuple[np.ndarray,
np.ndarray, np.ndarray]:
    """Compute macroscopic density and velocity from distribution
    functions.

    The macroscopic quantities are computed as moments of the
    distribution:
     $\rho = \sum f_i$ 
     $\rho u = \sum f_i * c_i$ 

    Args:
        f: Distribution functions (Nx, Ny, 9)

    Returns:
        rho: Density (Nx, Ny)
        u: x-velocity component (Nx, Ny)
        v: y-velocity component (Nx, Ny)

    Example:
    >>> lattice = D2Q9Lattice()
    >>> f = np.random.rand(10, 10, 9)
    >>> rho, u, v = lattice.compute_macroscopic(f)
    >>> rho.shape, u.shape, v.shape
    ((10, 10), (10, 10), (10, 10))
    """
    # Density: sum of all distributions
    rho = np.sum(f, axis=2)

    # Avoid division by zero
    rho_safe = np.where(rho > 1e-12, rho, 1.0)

    # Momentum: sum of distributions weighted by lattice velocities
    u = np.sum(f * self.c[:, 0], axis=2) / rho_safe
    v = np.sum(f * self.c[:, 1], axis=2) / rho_safe

    return rho, u, v

def __repr__(self) -> str:
    """String representation."""
    return (f"D2Q9Lattice(directions=9, cs={self.cs:.6f}, "
            f"weights_sum={np.sum(self.w):.6f})")

```

Utility functions for advanced usage

```
def compute_vorticity(u: np.ndarray, v: np.ndarray, dx: float = 1.0) -> np.ndarray:
    """Compute vorticity field:  $\omega = \partial v / \partial x - \partial u / \partial y$ .
```

Args:

```
    u: x-velocity component (Nx, Ny)
    v: y-velocity component (Nx, Ny)
    dx: Grid spacing (default: 1.0)
```

Returns:

```
    omega: Vorticity field (Nx, Ny)
    """
    dv_dx = np.gradient(v, dx, axis=0)
    du_dy = np.gradient(u, dx, axis=1)
    return dv_dx - du_dy
```

```
def compute_stream_function(u: np.ndarray, v: np.ndarray, dx: float = 1.0) -> np.ndarray:
    """Compute stream function  $\psi$  from velocity field.
```

The stream function satisfies: $u = \partial \psi / \partial y$, $v = -\partial \psi / \partial x$

Args:

```
    u: x-velocity component (Nx, Ny)
    v: y-velocity component (Nx, Ny)
    dx: Grid spacing (default: 1.0)
```

Returns:

```
    psi: Stream function (Nx, Ny)
```

Note:

```
    This uses a simple cumulative integration. For better accuracy,
    solve the Poisson equation:  $\nabla^2 \psi = \omega$ 
```

"""

```
# Simple integration approach (less accurate but fast)
psi = np.cumsum(u, axis=1) * dx
return psi
```

Verification in Jupyter Notebook

Create a new notebook with these cells:

Cell 1: Setup

```
import numpy as np
import matplotlib.pyplot as plt
from lattice import D2Q9Lattice, compute_vorticity

print("✓ Imports successful")
```

Cell 2: Test Lattice Initialization

```
print("="*60)
print("TEST 1: Lattice Initialization")
print("="*60)
```

```
lattice = D2Q9Lattice()
```

Check dimensions

```
print(f"\n✓ Lattice created: {lattice}")
print(f" Velocity directions shape: {lattice.c.shape}")
print(f" Weights shape: {lattice.w.shape}")
print(f" Speed of sound: cs = {lattice.cs:.6f}, cs2 = {lattice.cs2:.6f}")
```

Display velocity directions

```
print("\nVelocity directions:")
for i in range(9):
    print(f" c[{i}] = ({lattice.c[i,0]:2.0f}, {lattice.c[i,1]:2.0f}) "
          f"weight = {lattice.w[i]:.6f}")
```

Cell 3: Test Weight Normalization

```
print("\n" + "="*60)
print("TEST 2: Weight Normalization")
print("="*60)
```

```
weight_sum = np.sum(lattice.w)
error = abs(weight_sum - 1.0)
```

```
print(f"Sum of weights: {weight_sum:.15f}")
print(f"Error from 1.0: {error:.2e}")
print(f"Status: {'✓ PASS' if error < 1e-12 else '✗ FAIL'}")
```

Verify individual weight values

```
expected_weights = {
    0: 4/9, # rest
    1: 1/9, # cardinal
```

```

5: 1/36 # diagonal
}

print("\nIndividual weight verification:")
for idx, expected in expected_weights.items():
    actual = lattice.w[idx]
    error = abs(actual - expected)
    status = "✓" if error < 1e-12 else "✗"
    print(f" {status} w[{idx}] = {actual:.10f} (expected {expected:.10f})")

```

Cell 4: Test Opposite Directions

```

print("\n" + "="*60)
print("TEST 3: Opposite Direction Mapping")
print("="*60)

all_correct = True
print("\nOpposite direction pairs:")
for i in range(9):
    opp_i = lattice.opp[i]
    c_sum = lattice.c[i] + lattice.c[opp_i]
    is_opposite = np.allclose(c_sum, [0, 0])
    status = "✓" if is_opposite else "✗"
    all_correct = all_correct and is_opposite

print(f" {status} c[{i}] + c[{opp_i}] = "
      f"({lattice.c[i,0]:2.0f},{lattice.c[i,1]:2.0f}) + "
      f"({lattice.c[opp_i,0]:2.0f},{lattice.c[opp_i,1]:2.0f}) = "
      f"({c_sum[0]:.1f},{c_sum[1]:.1f})")

print(f"\nOverall: {'✓ PASS - All directions have correct opposites' if all_correct else '✗ FAIL'}")

```

Cell 5: Test Equilibrium Distribution

```

print("\n" + "="*60)
print("TEST 4: Equilibrium Distribution")
print("="*60)

```

Create uniform flow field

```

Nx, Ny = 10, 10
rho = np.ones((Nx, Ny))
u = 0.1 * np.ones((Nx, Ny))
v = 0.0 * np.ones((Nx, Ny))

```

Compute equilibrium

```

f_eq = lattice.equilibrium(rho, u, v)

```

```
print(f"Input field shape: ({Nx}, {Ny})")
print(f"Equilibrium distribution shape: {f_eq.shape}")
```

Test 1: Mass conservation

```
rho_from_eq = np.sum(f_eq, axis=2)
mass_error = np.max(np.abs(rho_from_eq - rho))

print(f"\nMass conservation:")
print(f" Input density: {rho[0,0]:.6f}")
print(f" Reconstructed density: {rho_from_eq[0,0]:.6f}")
print(f" Max error: {mass_error:.2e}")
print(f" Status: {'✓ PASS' if mass_error < 1e-12 else '✗ FAIL'}")
```

Test 2: Momentum conservation

```
u_from_eq = np.sum(f_eq * lattice.c[:, 0], axis=2) / rho_from_eq
v_from_eq = np.sum(f_eq * lattice.c[:, 1], axis=2) / rho_from_eq

u_error = np.max(np.abs(u_from_eq - u))
v_error = np.max(np.abs(v_from_eq - v))

print(f"\nMomentum conservation:")
print(f" Input velocity: u={u[0,0]:.6f}, v={v[0,0]:.6f}")
print(f" Reconstructed velocity: u={u_from_eq[0,0]:.6f}, v={v_from_eq[0,0]:.6f}")
print(f" Max error: u={u_error:.2e}, v={v_error:.2e}")
print(f" Status: {'✓ PASS' if max(u_error, v_error) < 1e-12 else '✗ FAIL'}")
```

Cell 6: Test Macroscopic Computation

```
print("\n" + "="*60)
print("TEST 5: Macroscopic Quantity Recovery")
print("="*60)
```

Start from equilibrium

```
rho_in = np.ones((Nx, Ny))
u_in = 0.05 * np.ones((Nx, Ny))
v_in = 0.02 * np.ones((Nx, Ny))
```

Compute equilibrium

```
f = lattice.equilibrium(rho_in, u_in, v_in)
```

Recover macroscopic quantities

```
rho_out, u_out, v_out = lattice.compute_macroscopic(f)
```

Compare

```
rho_error = np.max(np.abs(rho_out - rho_in))
u_error = np.max(np.abs(u_out - u_in))
v_error = np.max(np.abs(v_out - v_in))

print("Input → Equilibrium → Recovered:")
print(f" ρ: {rho_in[0,0]:.6f} → {rho_out[0,0]:.6f} (error: {rho_error:.2e})")
print(f" u: {u_in[0,0]:.6f} → {u_out[0,0]:.6f} (error: {u_error:.2e})")
print(f" v: {v_in[0,0]:.6f} → {v_out[0,0]:.6f} (error: {v_error:.2e})")

max_error = max(rho_error, u_error, v_error)
print(f"\nMax error: {max_error:.2e}")
print(f"Status: {'✓ PASS - Perfect recovery' if max_error < 1e-12 else '✗ FAIL'}")
```

Cell 7: Test Zero Velocity Equilibrium

```
print("\n" + "="*60)
print("TEST 6: Zero Velocity Equilibrium")
print("="*60)
```

At rest: $u=v=0$

```
rho_rest = np.ones((5, 5))
u_rest = np.zeros((5, 5))
v_rest = np.zeros((5, 5))

f_rest = lattice.equilibrium(rho_rest, u_rest, v_rest)
```

For zero velocity, equilibrium reduces to: $f_i^{eq} = w_i * \rho$

```
print("At rest (u=v=0), equilibrium should be:  $f_i = w_i * \rho$ ")
print("\nComparison for  $\rho=1$ :")
for i in range(9):
    expected = lattice.w[i] * rho_rest[0, 0]
    actual = f_rest[0, 0, i]
    error = abs(actual - expected)
```

```
status = "✓" if error < 1e-12 else "✗"  
print(f" {status} f[{i}] = {actual:.10f} (expected {expected:.10f})")
```

Cell 8: Visualize Equilibrium Distribution

```
print("\n" + "="*60)  
print("TEST 7: Equilibrium Distribution Visualization")  
print("="*60)
```

Create a flow with velocity

```
rho_vis = 1.0  
u_vis = 0.1  
v_vis = 0.05
```

Compute equilibrium at a single point

```
f_single = lattice.equilibrium(  
    np.array([[rho_vis]]),  
    np.array([[u_vis]]),  
    np.array([[v_vis]])  
)[0, 0, :]
```

Visualize

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
```

Plot 1: Distribution values

```
ax1.bar(range(9), f_single, color='steelblue', alpha=0.7)  
ax1.set_xlabel('Direction Index')  
ax1.set_ylabel('f_i')  
ax1.set_title(f'Equilibrium Distribution\n( $\rho$ = $\{rho\_vis\}$ ,  $u$ = $\{u\_vis\}$ ,  $v$ = $\{v\_vis\}$ )')  
ax1.grid(True, alpha=0.3)
```

Plot 2: Spatial distribution

```
ax2.set_xlim(-2, 2)  
ax2.set_ylim(-2, 2)  
ax2.set_aspect('equal')  
ax2.grid(True, alpha=0.3)  
ax2.set_xlabel('cx')
```

```
ax2.set_ylabel('cy')
ax2.set_title('Distribution in Velocity Space')
```

Plot each direction as a circle sized by distribution value

```
for i in range(9):
    cx, cy = lattice.c[i]
    size = f_single[i] * 5000 # Scale for visibility
    ax2.scatter(cx, cy, s=size, alpha=0.6,
    label=f'f[{i}]={f_single[i]:.4f}')
    ax2.annotate(f'{i}', (cx, cy), ha='center', va='center', fontsize=8)
```

Add velocity vector

```
ax2.arrow(0, 0, u_vis10, v_vis10, head_width=0.3, head_length=0.3,
fc='red', ec='red', linewidth=2, label='Velocity')
ax2.legend(bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=8)
```

```
plt.tight_layout()
plt.show()
```

```
print("✓ Visualization complete")
print(" Larger circles = higher distribution values")
print(" Red arrow = macroscopic velocity direction")
```

Cell 9: Test Vorticity Computation

```
print("\n" + "="*60)
print("TEST 8: Vorticity Computation")
print("="*60)
```

Create a simple vortex velocity field

```
x = np.linspace(-1, 1, 20)
y = np.linspace(-1, 1, 20)
X, Y = np.meshgrid(x, y, indexing='ij')
```

Circular vortex: $u = -y$, $v = x$

```
u_vortex = -Y
v_vortex = X
```

Compute vorticity

```
omega = compute_vorticity(u_vortex, v_vortex, dx=x[1]-x[0])
```

```
print(f"Velocity field shape: {u_vortex.shape}")  
print(f"Vorticity field shape: {omega.shape}")
```

For this exact vortex, $\omega =$
 $\partial v / \partial x - \partial u / \partial y = 1 - (-1) = 2$
everywhere

```
print(f"\nTheoretical vorticity: 2.0 (everywhere)")  
print(f"Computed vorticity (center): {omega[10, 10]:.4f}")  
print(f"Mean vorticity: {np.mean(omega):.4f}")
```

Visualize

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
```

Velocity field

```
ax = axes[0]  
ax.quiver(X[::2, ::2], Y[::2, ::2], u_vortex[::2, ::2], v_vortex[::2, ::2])  
ax.set_title('Velocity Field')  
ax.set_xlabel('x')  
ax.set_ylabel('y')  
ax.set_aspect('equal')
```

Vorticity

```
ax = axes[1]  
im = ax.contourf(X, Y, omega, levels=20, cmap='RdBu_r')  
plt.colorbar(im, ax=ax, label='ω')  
ax.set_title('Vorticity Field')  
ax.set_xlabel('x')  
ax.set_ylabel('y')  
ax.set_aspect('equal')
```

Vorticity histogram

```

ax = axes[2]
ax.hist(omega.flatten(), bins=30, edgecolor='black', alpha=0.7)
ax.axvline(2.0, color='red', linestyle='--', label='Theoretical ( $\omega=2$ )')
ax.set_xlabel('Vorticity')
ax.set_ylabel('Frequency')
ax.set_title('Vorticity Distribution')
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("✓ Vorticity computation working correctly")

```

Cell 10: Summary

```

print("\n" + "="*60)
print("VERIFICATION SUMMARY")
print("="*60)

```

```

summary = """
✓ All lattice tests completed!

```

The D2Q9Lattice component provides:

1. ✓ 9 velocity directions with correct weights
2. ✓ Weight normalization ($\sum w_i = 1$)
3. ✓ Opposite direction mapping for bounce-back
4. ✓ Equilibrium distribution computation
5. ✓ Macroscopic quantity recovery (ρ, u, v)
6. ✓ Mass and momentum conservation
7. ✓ Vorticity computation utility

Properties Verified:

- Weights sum to $1.0 \pm 1e-12$
- Opposite directions cancel: $c[i] + c[\text{opp}[i]] = (0,0)$
- Equilibrium preserves mass: $\sum f_i^{\text{eq}} = \rho$
- Equilibrium preserves momentum: $\sum f_i^{\text{eq}} \cdot c_i = \rho u$
- Zero velocity gives $f_i^{\text{eq}} = w_i \cdot \rho$

Next Steps:

- Build geometry_builder.py (obstacle generation)
- Build lbm_core.py (collision and streaming)
- Integrate with config_manager.py

```

Run unit tests with: pytest tests/test_lattice.py -v
"""

```

```
print(summary)
```

Unit Test File (Optional)

Create `tests/test_lattice.py`:

```
"""Unit tests for D2Q9 lattice."""
```

```
import numpy as np
import pytest
from lattice import D2Q9Lattice, compute_vorticity
```

```
def test_lattice_initialization():
    """Test lattice initializes correctly."""
    lattice = D2Q9Lattice()
```

```
    assert lattice.c.shape == (9, 2)
    assert lattice.w.shape == (9,)
    assert lattice.opp.shape == (9,)
    assert abs(lattice.cs2 - 1/3) < 1e-12
```

```
def test_weight_normalization():
    """Test weights sum to 1."""
    lattice = D2Q9Lattice()
    assert abs(np.sum(lattice.w) - 1.0) < 1e-12
```

```
def test_opposite_directions():
    """Test opposite direction mapping."""
    lattice = D2Q9Lattice()
```

```
    for i in range(9):
        opp_i = lattice.opp[i]
        assert np.allclose(lattice.c[i], -lattice.c[opp_i])
```

```
def test_equilibrium_mass_conservation():
    """Test equilibrium preserves mass."""
    lattice = D2Q9Lattice()
```

```
    rho = np.ones((10, 10))
    u = 0.1 * np.ones((10, 10))
    v = 0.0 * np.ones((10, 10))
```

```
    f_eq = lattice.equilibrium(rho, u, v)
    rho_recovered = np.sum(f_eq, axis=2)
```

```
    assert np.allclose(rho_recovered, rho, atol=1e-12)
```

```
def test_macroscopic_recovery():
    """Test macroscopic quantities can be recovered."""
    lattice = D2Q9Lattice()
```

```

rho_in = np.ones((10, 10))
u_in = 0.05 * np.ones((10, 10))
v_in = 0.02 * np.ones((10, 10))

f = lattice.equilibrium(rho_in, u_in, v_in)
rho_out, u_out, v_out = lattice.compute_macroscopic(f)

assert np.allclose(rho_out, rho_in, atol=1e-12)
assert np.allclose(u_out, u_in, atol=1e-12)
assert np.allclose(v_out, v_in, atol=1e-12)

def test_zero_velocity_equilibrium():
    """Test equilibrium at rest."""
    lattice = D2Q9Lattice()

    rho = np.ones((5, 5))
    u = np.zeros((5, 5))
    v = np.zeros((5, 5))

    f_eq = lattice.equilibrium(rho, u, v)

    # At rest:  $f_i^{eq} = w_i * \rho$ 
    for i in range(9):
        expected = lattice.w[i] * rho
        assert np.allclose(f_eq[:, :, i], expected, atol=1e-12)

def test_vorticity_computation():
    """Test vorticity computation for known field."""
    # Uniform flow (no vorticity)
    u = np.ones((10, 10))
    v = np.zeros((10, 10))
    omega = compute_vorticity(u, v)

    # Should be approximately zero (numerical gradients)
    assert np.max(np.abs(omega)) < 0.1

if __name__ == '__main__':
    pytest.main([__file__, '-v'])

```

Expected Output

When you run the verification notebook, you should see:

- ✓ Lattice initialized with 9 directions
- ✓ Weights sum to 1.0000000000000000 (error < 1e-12)
- ✓ All 9 opposite directions verified
- ✓ Mass conservation: max error < 1e-12
- ✓ Momentum conservation: max error < 1e-12
- ✓ Zero velocity equilibrium correct
- ✓ Vorticity computation working

Success Criteria

- ✓ Lattice initializes without errors
- ✓ Weights sum to exactly 1.0
- ✓ All opposite directions verified
- ✓ Equilibrium preserves mass (error < 1e-12)
- ✓ Equilibrium preserves momentum (error < 1e-12)
- ✓ Zero velocity gives $f_i = w_i * \rho$
- ✓ Vorticity computation produces reasonable values

Integration with Config Manager

The lattice will be used in the LBM core like this:

```
from config_manager import ConfigManager
from lattice import D2Q9Lattice
```

Load configuration

```
config = ConfigManager('examples/cylinder_re100.yaml')
```

Initialize lattice

```
lattice = D2Q9Lattice()
```

Use lattice properties

```
cs2 = lattice.cs2
tau = config.get('derived.tau')

print(f"Relaxation time: {tau:.6f}")
print(f"Speed of sound squared: {cs2:.6f}")
```

Once these tests pass, you're ready for the next component!

Building the Lattice Manager - Second Component

Overview

The Lattice Manager defines the D2Q9 (2D, 9-velocity) lattice structure, which is the foundation of the Lattice Boltzmann Method. This component provides:

- 9 lattice velocity directions
- Corresponding weights
- Opposite direction mapping (for bounce-back)
- Equilibrium distribution computation
- Macroscopic quantity extraction

File to Create: **lattice.py**

```
"""D2Q9 Lattice for 2D Lattice Boltzmann Method.
```

```
This module defines the D2Q9 lattice structure with 9 velocity directions,
providing equilibrium distributions and macroscopic quantity computations.
"""
```

```
import numpy as np
from typing import Tuple
```

```
class D2Q9Lattice:
    """D2Q9 lattice for 2D Lattice Boltzmann Method.
```

```
The D2Q9 lattice has 9 velocity directions:
```

- ```
- 1 rest particle (0,0)
- 4 cardinal directions ($\pm 1, 0$) and ($0, \pm 1$)
- 4 diagonal directions ($\pm 1, \pm 1$)
```

```
Attributes:
```

```
 c: Lattice velocities (9, 2) - [x, y] components
 w: Lattice weights (9,) - normalized to sum to 1
 opp: Opposite directions (9,) - indices for bounce-back
 cs: Speed of sound ($1/\sqrt{3}$)
 cs2: Speed of sound squared ($1/3$)
 """
```

```
def __init__(self):
 """Initialize D2Q9 lattice structure."""
 # Lattice velocities (9 directions)
 # Index layout:
 # 6 2 5
 # ↖ ↑ ↗
 # 3 ← 0 → 1
 # ↙ ↓ ↘
 # 7 4 8
 self.c = np.array([
 [0, 0], # 0: rest
 [1, 0], # 1: east
 [0, 1], # 2: north
 [-1, 0], # 3: west
 [0, -1], # 4: south
 [1, 1], # 5: northeast
```

```

 [-1, 1], # 6: northwest
 [-1,-1], # 7: southwest
 [1, -1] # 8: southeast
], dtype=float)

 # Lattice weights (normalized)
 # w[0] = 4/9 for rest particle
 # w[1-4] = 1/9 for cardinal directions
 # w[5-8] = 1/36 for diagonal directions
 self.w = np.array([
 4/9, # 0: rest
 1/9, # 1-4: cardinal
 1/9,
 1/9,
 1/9,
 1/36, # 5-8: diagonal
 1/36,
 1/36,
 1/36
])

 # Opposite directions (for bounce-back boundary condition)
 # opp[i] gives the index of the direction opposite to i
 self.opp = np.array([0, 3, 4, 1, 2, 7, 8, 5, 6])

 # Speed of sound in lattice units
 self.cs = 1.0 / np.sqrt(3.0)
 self.cs2 = self.cs**2

 # Verify lattice properties
 self._verify_lattice_properties()

def _verify_lattice_properties(self):
 """Verify lattice satisfies required properties."""
 # Check weights sum to 1
 weight_sum = np.sum(self.w)
 assert abs(weight_sum - 1.0) < 1e-12, f"Weights sum to {weight_sum}, not 1.0"

 # Check opposite directions
 for i in range(9):
 opp_i = self.opp[i]
 assert np.allclose(self.c[i], -self.c[opp_i]), \
 f"Direction {i} and opposite {opp_i} don't cancel"

 print("✓ D2Q9 lattice properties verified")

def equilibrium(self, rho: np.ndarray, u: np.ndarray, v: np.ndarray) -> np.ndarray:
 """Compute equilibrium distribution function.
```

The equilibrium distribution is given by:

$$f_i^{\text{eq}} = w_i * \rho * [1 + (c_i \cdot u) / cs^2 + (c_i \cdot u)^2 / (2cs^4) - u^2 / (2cs^2)]$$

Args:

rho: Density field (Nx, Ny)  
u: x-velocity component (Nx, Ny)  
v: y-velocity component (Nx, Ny)

Returns:

f\_eq: Equilibrium distributions (Nx, Ny, 9)

Example:

```
>>> lattice = D2Q9Lattice()
>>> rho = np.ones((10, 10))
>>> u = 0.1 * np.ones((10, 10))
>>> v = np.zeros((10, 10))
>>> f_eq = lattice.equilibrium(rho, u, v)
>>> f_eq.shape
(10, 10, 9)
"""
Nx, Ny = rho.shape
f_eq = np.zeros((Nx, Ny, 9))

Velocity squared
u2 = u**2 + v**2

Compute equilibrium for each direction
for i in range(9):
 # Lattice velocity dotted with macroscopic velocity
 cu = self.c[i, 0] * u + self.c[i, 1] * v

 # Equilibrium distribution
 f_eq[:, :, i] = self.w[i] * rho * (
 1.0
 + cu / self.cs2
 + cu**2 / (2 * self.cs2**2)
 - u2 / (2 * self.cs2)
)

return f_eq

def compute_macroscopic(self, f: np.ndarray) -> Tuple[np.ndarray,
np.ndarray, np.ndarray]:
 """Compute macroscopic density and velocity from distribution
 functions.

 The macroscopic quantities are computed as moments of the
 distribution:
 $\rho = \sum f_i$
```

```

 pu = $\sum f_i * c_i$

 Args:
 f: Distribution functions (Nx, Ny, 9)

 Returns:
 rho: Density (Nx, Ny)
 u: x-velocity component (Nx, Ny)
 v: y-velocity component (Nx, Ny)

 Example:
 >>> lattice = D2Q9Lattice()
 >>> f = np.random.rand(10, 10, 9)
 >>> rho, u, v = lattice.compute_macroscopic(f)
 >>> rho.shape, u.shape, v.shape
 ((10, 10), (10, 10), (10, 10))
 """
 # Density: sum of all distributions
 rho = np.sum(f, axis=2)

 # Avoid division by zero
 rho_safe = np.where(rho > 1e-12, rho, 1.0)

 # Momentum: sum of distributions weighted by lattice velocities
 u = np.sum(f * self.c[:, 0], axis=2) / rho_safe
 v = np.sum(f * self.c[:, 1], axis=2) / rho_safe

 return rho, u, v

def __repr__(self) -> str:
 """String representation."""
 return (f"D2Q9Lattice(directions=9, cs={self.cs:.6f}, "
 f"weights_sum={np.sum(self.w):.6f})")

```

## Utility functions for advanced usage

```

def compute_vorticity(u: np.ndarray, v: np.ndarray, dx: float = 1.0) -> np.ndarray:
 """Compute vorticity field: $\omega = \partial v / \partial x - \partial u / \partial y$.

```

```

 Args:
 u: x-velocity component (Nx, Ny)
 v: y-velocity component (Nx, Ny)
 dx: Grid spacing (default: 1.0)

```

```

 Returns:

```

```

 omega: Vorticity field (Nx, Ny)
"""
dv_dx = np.gradient(v, dx, axis=0)
du_dy = np.gradient(u, dx, axis=1)
return dv_dx - du_dy

def compute_stream_function(u: np.ndarray, v: np.ndarray, dx: float = 1.0) -> np.ndarray:
 """Compute stream function ψ from velocity field.

 The stream function satisfies: $u = \partial\psi/\partial y$, $v = -\partial\psi/\partial x$

 Args:
 u: x-velocity component (Nx, Ny)
 v: y-velocity component (Nx, Ny)
 dx: Grid spacing (default: 1.0)

 Returns:
 psi: Stream function (Nx, Ny)

 Note:
 This uses a simple cumulative integration. For better accuracy,
 solve the Poisson equation: $\nabla^2 \psi = \omega$
 """
 # Simple integration approach (less accurate but fast)
 psi = np.cumsum(u, axis=1) * dx
 return psi

```

## Verification in Jupyter Notebook

Create a new notebook with these cells:

### Cell 1: Setup

```

import numpy as np
import matplotlib.pyplot as plt
from lattice import D2Q9Lattice, compute_vorticity

print("✓ Imports successful")

```

### Cell 2: Test Lattice Initialization

```

print("="*60)
print("TEST 1: Lattice Initialization")
print("="*60)

lattice = D2Q9Lattice()

```

## Check dimensions

```

print(f"\n✓ Lattice created: {lattice}")
print(f" Velocity directions shape: {lattice.c.shape}")
print(f" Weights shape: {lattice.w.shape}")
print(f" Speed of sound: cs = {lattice.cs:.6f}, cs2 = {lattice.cs2:.6f}")

```

## Display velocity directions

```

print("\nVelocity directions:")
for i in range(9):
 print(f" c[{i}] = ({lattice.c[i,0]:2.0f}, {lattice.c[i,1]:2.0f}) "
 f"weight = {lattice.w[i]:.6f}")

```

### Cell 3: Test Weight Normalization

```

print("\n" + "="*60)
print("TEST 2: Weight Normalization")
print("="*60)

weight_sum = np.sum(lattice.w)
error = abs(weight_sum - 1.0)

print(f"Sum of weights: {weight_sum:.15f}")
print(f"Error from 1.0: {error:.2e}")
print(f"Status: {'✓ PASS' if error < 1e-12 else '✗ FAIL'}")

```

## Verify individual weight values

```

expected_weights = {
 0: 4/9, # rest
 1: 1/9, # cardinal
 5: 1/36 # diagonal
}

print("\nIndividual weight verification:")
for idx, expected in expected_weights.items():
 actual = lattice.w[idx]
 error = abs(actual - expected)
 status = "✓" if error < 1e-12 else "✗"
 print(f" {status} w[{idx}] = {actual:.10f} (expected {expected:.10f})")

```

### Cell 4: Test Opposite Directions

```

print("\n" + "="*60)
print("TEST 3: Opposite Direction Mapping")
print("="*60)

```

```

all_correct = True
print("\nOpposite direction pairs:")
for i in range(9):
 opp_i = lattice.opp[i]
 c_sum = lattice.c[i] + lattice.c[opp_i]
 is_opposite = np.allclose(c_sum, [0, 0])
 status = "✓" if is_opposite else "✗"
 all_correct = all_correct and is_opposite

print(f" {status} c[{i}] + c[{opp_i}] = "
 f"({lattice.c[i,0]:2.0f},{lattice.c[i,1]:2.0f}) + "
 f"({lattice.c[opp_i,0]:2.0f},{lattice.c[opp_i,1]:2.0f}) = "
 f"({c_sum[0]:.1f},{c_sum[1]:.1f})")

print(f"\nOverall: {'✓ PASS - All directions have correct opposites' if all_correct else '✗ FAIL'}")

```

## Cell 5: Test Equilibrium Distribution

```

print("\n" + "="*60)
print("TEST 4: Equilibrium Distribution")
print("="*60)

```

# Create uniform flow field

```

Nx, Ny = 10, 10
rho = np.ones((Nx, Ny))
u = 0.1 * np.ones((Nx, Ny))
v = 0.0 * np.ones((Nx, Ny))

```

# Compute equilibrium

```

f_eq = lattice.equilibrium(rho, u, v)

print(f"Input field shape: ({Nx}, {Ny})")
print(f"Equilibrium distribution shape: {f_eq.shape}")

```

# Test 1: Mass conservation

```

rho_from_eq = np.sum(f_eq, axis=2)
mass_error = np.max(np.abs(rho_from_eq - rho))

print(f"\nMass conservation:")
print(f" Input density: {rho[0,0]:.6f}")
print(f" Reconstructed density: {rho_from_eq[0,0]:.6f}")
print(f" Max error: {mass_error:.2e}")
print(f" Status: {'✓ PASS' if mass_error < 1e-12 else '✗ FAIL'}")

```

# Test 2: Momentum conservation

```
u_from_eq = np.sum(f_eq * lattice.c[:, 0], axis=2) / rho_from_eq
v_from_eq = np.sum(f_eq * lattice.c[:, 1], axis=2) / rho_from_eq

u_error = np.max(np.abs(u_from_eq - u))
v_error = np.max(np.abs(v_from_eq - v))

print(f"\nMomentum conservation:")
print(f" Input velocity: u={u[0,0]:.6f}, v={v[0,0]:.6f}")
print(f" Reconstructed velocity: u={u_from_eq[0,0]:.6f}, v={v_from_eq[0,0]:.6f}")
print(f" Max error: u={u_error:.2e}, v={v_error:.2e}")
print(f" Status: {'✓ PASS' if max(u_error, v_error) < 1e-12 else '✗ FAIL'}")
```

## Cell 6: Test Macroscopic Computation

```
print("\n" + "="*60)
print("TEST 5: Macroscopic Quantity Recovery")
print("="*60)
```

# Start from equilibrium

```
rho_in = np.ones((Nx, Ny))
u_in = 0.05 * np.ones((Nx, Ny))
v_in = 0.02 * np.ones((Nx, Ny))
```

# Compute equilibrium

```
f = lattice.equilibrium(rho_in, u_in, v_in)
```

# Recover macroscopic quantities

```
rho_out, u_out, v_out = lattice.compute_macroscopic(f)
```

# Compare

```
rho_error = np.max(np.abs(rho_out - rho_in))
u_error = np.max(np.abs(u_out - u_in))
v_error = np.max(np.abs(v_out - v_in))
```

```

print("Input → Equilibrium → Recovered:")
print(f" ρ: {rho_in[0,0]:.6f} → {rho_out[0,0]:.6f} (error: {rho_error:.2e})")
print(f" u: {u_in[0,0]:.6f} → {u_out[0,0]:.6f} (error: {u_error:.2e})")
print(f" v: {v_in[0,0]:.6f} → {v_out[0,0]:.6f} (error: {v_error:.2e})")

max_error = max(rho_error, u_error, v_error)
print(f"\nMax error: {max_error:.2e}")
print(f"Status: {'✓ PASS - Perfect recovery' if max_error < 1e-12 else '✗ FAIL'}")

```

## Cell 7: Test Zero Velocity Equilibrium

```

print("\n" + "="*60)
print("TEST 6: Zero Velocity Equilibrium")
print("="*60)

```

# At rest: $u=v=0$

```

rho_rest = np.ones((5, 5))
u_rest = np.zeros((5, 5))
v_rest = np.zeros((5, 5))

f_rest = lattice.equilibrium(rho_rest, u_rest, v_rest)

```

# For zero velocity, equilibrium reduces to: $f_i^{eq} = w_i * \rho$

```

print("At rest (u=v=0), equilibrium should be: f_i = w_i * ρ")
print("\nComparison for ρ=1:")
for i in range(9):
 expected = lattice.w[i] * rho_rest[0, 0]
 actual = f_rest[0, 0, i]
 error = abs(actual - expected)
 status = "✓" if error < 1e-12 else "✗"
 print(f" {status} f[{i}] = {actual:.10f} (expected {expected:.10f})")

```

## Cell 8: Visualize Equilibrium Distribution

```

print("\n" + "="*60)
print("TEST 7: Equilibrium Distribution Visualization")
print("="*60)

```

# Create a flow with velocity

```

rho_vis = 1.0
u_vis = 0.1
v_vis = 0.05

```

# Compute equilibrium at a single point

```
f_single = lattice.equilibrium(
 np.array([[rho_vis]]),
 np.array([[u_vis]]),
 np.array([[v_vis]])
)[0, 0, :]
```

## Visualize

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
```

### Plot 1: Distribution values

```
ax1.bar(range(9), f_single, color='steelblue', alpha=0.7)
ax1.set_xlabel('Direction Index')
ax1.set_ylabel('fi')
ax1.set_title(f'Equilibrium Distribution\n(ρ = $\{rho_vis\}$, u = $\{u_vis\}$, v = $\{v_vis\}$)')
ax1.grid(True, alpha=0.3)
```

### Plot 2: Spatial distribution

```
ax2.set_xlim(-2, 2)
ax2.set_ylim(-2, 2)
ax2.set_aspect('equal')
ax2.grid(True, alpha=0.3)
ax2.set_xlabel('cx')
ax2.set_ylabel('cy')
ax2.set_title('Distribution in Velocity Space')
```

### Plot each direction as a circle sized by distribution value

```
for i in range(9):
 cx, cy = lattice.c[i]
 size = f_single[i] * 5000 # Scale for visibility
 ax2.scatter(cx, cy, s=size, alpha=0.6,
 label=f'f{i}'=f'f{i}:.4f')
 ax2.annotate(f'f{i}', (cx, cy), ha='center', va='center', fontsize=8)
```

# Add velocity vector

```
ax2.arrow(0, 0, u_vis10, v_vis10, head_width=0.3, head_length=0.3,
fc='red', ec='red', linewidth=2, label='Velocity')
ax2.legend(bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=8)
```

```
plt.tight_layout()
plt.show()
```

```
print("✓ Visualization complete")
print("Larger circles = higher distribution values")
print("Red arrow = macroscopic velocity direction")
```

## Cell 9: Test Vorticity Computation

```
print("\n" + "="*60)
print("TEST 8: Vorticity Computation")
print("="*60)
```

# Create a simple vortex velocity field

```
x = np.linspace(-1, 1, 20)
y = np.linspace(-1, 1, 20)
X, Y = np.meshgrid(x, y, indexing='ij')
```

## Circular vortex: $u = -y$ , $v = x$

```
u_vortex = -Y
v_vortex = X
```

## Compute vorticity

```
omega = compute_vorticity(u_vortex, v_vortex, dx=x[1]-x[0])
```

```
print(f"Velocity field shape: {u_vortex.shape}")
print(f"Vorticity field shape: {omega.shape}")
```

**For this exact vortex,  $\omega =$   
 $\partial v / \partial x - \partial u / \partial y = 1 - (-1) = 2$   
everywhere**

```
print(f"\nTheoretical vorticity: 2.0 (everywhere)")
print(f"Computed vorticity (center): {omega[10, 10]:.4f}")
print(f"Mean vorticity: {np.mean(omega):.4f}")
```

## Visualize

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
```

## Velocity field

```
ax = axes[0]
ax.quiver(X[:, :2], Y[:, :2], u_vortex[:, :2], v_vortex[:, :2])
ax.set_title('Velocity Field')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_aspect('equal')
```

## Vorticity

```
ax = axes[1]
im = ax.contourf(X, Y, omega, levels=20, cmap='RdBu_r')
plt.colorbar(im, ax=ax, label='ω')
ax.set_title('Vorticity Field')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_aspect('equal')
```

## Vorticity histogram

```
ax = axes[2]
ax.hist(omega.flatten(), bins=30, edgecolor='black', alpha=0.7)
ax.axvline(2.0, color='red', linestyle='--', label='Theoretical (ω=2)')
ax.set_xlabel('Vorticity')
ax.set_ylabel('Frequency')
ax.set_title('Vorticity Distribution')
ax.legend()
ax.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```

```
print("✓ Vorticity computation working correctly")
```

## Cell 10: Summary

```
print("\n" + "="*60)
print("VERIFICATION SUMMARY")
print("="*60)
```

```
summary = """
```

```
✓ All lattice tests completed!
```

The D2Q9Lattice component provides:

1. ✓ 9 velocity directions with correct weights
2. ✓ Weight normalization ( $\sum w_i = 1$ )
3. ✓ Opposite direction mapping for bounce-back
4. ✓ Equilibrium distribution computation
5. ✓ Macroscopic quantity recovery ( $\rho, u, v$ )
6. ✓ Mass and momentum conservation
7. ✓ Vorticity computation utility

Properties Verified:

- Weights sum to  $1.0 \pm 1e-12$
- Opposite directions cancel:  $c[i] + c[\text{opp}[i]] = (0,0)$
- Equilibrium preserves mass:  $\sum f_i^{\text{eq}} = \rho$
- Equilibrium preserves momentum:  $\sum f_i^{\text{eq}} \cdot c_i = \rho u$
- Zero velocity gives  $f_i^{\text{eq}} = w_i \cdot \rho$

Next Steps:

- Build geometry\_builder.py (obstacle generation)
- Build lbm\_core.py (collision and streaming)
- Integrate with config\_manager.py

Run unit tests with: `pytest tests/test_lattice.py -v`

```
"""
```

```
print(summary)
```

## Unit Test File (Optional)

Create `tests/test_lattice.py`:

```
"""Unit tests for D2Q9 lattice."""
```

```

import numpy as np
import pytest
from lattice import D2Q9Lattice, compute_vorticity

def test_lattice_initialization():
 """Test lattice initializes correctly."""
 lattice = D2Q9Lattice()

 assert lattice.c.shape == (9, 2)
 assert lattice.w.shape == (9,)
 assert lattice.opp.shape == (9,)
 assert abs(lattice.cs2 - 1/3) < 1e-12

def test_weight_normalization():
 """Test weights sum to 1."""
 lattice = D2Q9Lattice()
 assert abs(np.sum(lattice.w) - 1.0) < 1e-12

def test_opposite_directions():
 """Test opposite direction mapping."""
 lattice = D2Q9Lattice()

 for i in range(9):
 opp_i = lattice.opp[i]
 assert np.allclose(lattice.c[i], -lattice.c[opp_i])

def test_equilibrium_mass_conservation():
 """Test equilibrium preserves mass."""
 lattice = D2Q9Lattice()

 rho = np.ones((10, 10))
 u = 0.1 * np.ones((10, 10))
 v = 0.0 * np.ones((10, 10))

 f_eq = lattice.equilibrium(rho, u, v)
 rho_recovered = np.sum(f_eq, axis=2)

 assert np.allclose(rho_recovered, rho, atol=1e-12)

def test_macroscopic_recovery():
 """Test macroscopic quantities can be recovered."""
 lattice = D2Q9Lattice()

 rho_in = np.ones((10, 10))
 u_in = 0.05 * np.ones((10, 10))
 v_in = 0.02 * np.ones((10, 10))

 f = lattice.equilibrium(rho_in, u_in, v_in)
 rho_out, u_out, v_out = lattice.compute_macroscopic(f)

 assert np.allclose(rho_out, rho_in, atol=1e-12)

```

```

assert np.allclose(u_out, u_in, atol=1e-12)
assert np.allclose(v_out, v_in, atol=1e-12)

def test_zero_velocity_equilibrium():
 """Test equilibrium at rest."""
 lattice = D2Q9Lattice()

 rho = np.ones((5, 5))
 u = np.zeros((5, 5))
 v = np.zeros((5, 5))

 f_eq = lattice.equilibrium(rho, u, v)

 # At rest: $f_i^{eq} = w_i * rho$
 for i in range(9):
 expected = lattice.w[i] * rho
 assert np.allclose(f_eq[:, :, i], expected, atol=1e-12)

def test_vorticity_computation():
 """Test vorticity computation for known field."""
 # Uniform flow (no vorticity)
 u = np.ones((10, 10))
 v = np.zeros((10, 10))
 omega = compute_vorticity(u, v)

 # Should be approximately zero (numerical gradients)
 assert np.max(np.abs(omega)) < 0.1

if __name__ == '__main__':
 pytest.main([__file__, '-v'])

```

## Expected Output

When you run the verification notebook, you should see:

- ✓ Lattice initialized with 9 directions
- ✓ Weights sum to 1.0000000000000000 (error < 1e-12)
- ✓ All 9 opposite directions verified
- ✓ Mass conservation: max error < 1e-12
- ✓ Momentum conservation: max error < 1e-12
- ✓ Zero velocity equilibrium correct
- ✓ Vorticity computation working

## Success Criteria

- ✓ Lattice initializes without errors
- ✓ Weights sum to exactly 1.0
- ✓ All opposite directions verified
- ✓ Equilibrium preserves mass (error < 1e-12)
- ✓ Equilibrium preserves momentum (error < 1e-12)

- ✓ Zero velocity gives  $f_i = w_i * \rho$
- ✓ Vorticity computation produces reasonable values

## Integration with Config Manager

The lattice will be used in the LBM core like this:

```
from config_manager import ConfigManager
from lattice import D2Q9Lattice
```

## Load configuration

```
config = ConfigManager('examples/cylinder_re100.yaml')
```

## Initialize lattice

```
lattice = D2Q9Lattice()
```

## Use lattice properties

```
cs2 = lattice.cs2
tau = config.get('derived.tau')

print(f"Relaxation time: {tau:.6f}")
print(f"Speed of sound squared: {cs2:.6f}")
```

Once these tests pass, you're ready for the next component!